

Mission 1

1

- Bringing the stack up: `docker compose up -d`

Explanation: This is the entry point of everything, we start our containers (kibana, elasticsearch, kibana and logstash).

- Starting the generator

```
MINGW64:/c:/Users/PC/Workspace/ELK/lab_1/elk-stack-coursework/course-2-logstash/lab2/generator
PC@DESKTOP-SJHCAIT MINGW64 ~
$ cd C:\Users\PC\Workspace\ELK\lab_1\elk-stack-coursework\course-2-logstash\lab2
bash: cd: C:\Users\PC\Workspace\ELK\lab_1\elk-stack-coursework\course-2-logstash\lab2: No such file or directory

PC@DESKTOP-SJHCAIT MINGW64 ~
$ cd "C:\Users\PC\Workspace\ELK\lab_1\elk-stack-coursework\course-2-logstash\lab2"

PC@DESKTOP-SJHCAIT MINGW64 ~/Workspace/ELK/lab_1/elk-stack-coursework/course-2-logstash/lab2 (main)
$ cd generator/

PC@DESKTOP-SJHCAIT MINGW64 ~/Workspace/ELK/lab_1/elk-stack-coursework/course-2-logstash/lab2/generator (main)
$ python generate_logs.py
100 events...
200 events...
300 events...
400 events...
500 events...
600 events...
700 events...
800 events...
900 events...
1000 events...
```

- Confirmation of raw events from all three sources:

Command: `docker compose logs -f logstash`

```
lab2-logstash | {
lab2-logstash |     "@version" => "1",
lab2-logstash |     "path" => "/var/log/shopfast/app.log",
lab2-logstash |     "tags" => [
lab2-logstash |         [0] "app"
lab2-logstash |     ],
lab2-logstash |     "host" => "819c4dc4772d",
lab2-logstash |     "@timestamp" => 2026-07-28T20:19:36.984256847Z,
lab2-logstash |     "message" => "2026-07-28T20:19:36Z|checkout|WARN|order=17164|amount=380.28|msg=payment failed, cart preserved\r"
lab2-logstash | }
lab2-logstash | {
lab2-logstash |     "@version" => "1",
lab2-logstash |     "path" => "/var/log/shopfast/access.log",
lab2-logstash |     "tags" => [
lab2-logstash |         [0] "web"
lab2-logstash |     ],
lab2-logstash |     "host" => "819c4dc4772d",
lab2-logstash |     "@timestamp" => 2026-07-28T20:19:37.160342506Z,
lab2-logstash |     "message" => "200.160.2.3 - - [28/Jul/2026:20:19:37 +0000] \"GET /product/887 HTTP/1.1\" 200 9348 \"https://twitter.com/\" \"cur1/8.5.0\" \r"
lab2-logstash | }
lab2-logstash | {
lab2-logstash |     "@version" => "1",
lab2-logstash |     "path" => "/var/log/shopfast/orders.csv",
lab2-logstash |     "tags" => [
lab2-logstash |         [0] "orders"
lab2-logstash |     ],
lab2-logstash |     "host" => "819c4dc4772d",
lab2-logstash |     "@timestamp" => 2026-07-28T20:19:37.162319253Z,
lab2-logstash |     "message" => "67"
```

2.

- Restarting the logstash container: `docker compose restart logstash`

Event with message containing specific **orderID** 79819. We'll check the order 79819 before and after restarting **Logstash** container.

```

lab2-logstash | [0] "app"
lab2-logstash | ],
lab2-logstash | "host" => "819c4dc4772d",
lab2-logstash | "@timestamp" => 2026-07-28T19:35:35.075679610Z,
lab2-logstash | "message" => "2026-07-28T19:35:35Z|checkout|INFO|order=79819|amount=199.35|msg=checkout started\r"
lab2-logstash | }
lab2-logstash | {
lab2-logstash | "version" => "1"

```

Before restarting Logstash container:

Command: `docker compose logs logstash | grep -F 'order=79819'`

```

PC@DESKTOP-SJHCAIT MINGW64 ~/Workspace/ELK/lab_1/elk-stack-coursework/course-2-logstash/lab2 (main)
$ docker compose logs logstash | grep -F 'order=79819'
lab2-logstash | "message" => "2026-07-28T19:35:35Z|checkout|INFO|order=79819|amount=199.35|msg=checkout started\r"
lab2-logstash | "message" => "2026-07-28T21:47:32Z|checkout|INFO|order=79819|amount=199.35|msg=checkout started\r"

```

Checking message with order **79819** After restarting Logstash container:

```

PC@DESKTOP-SJHCAIT MINGW64 ~/Workspace/ELK/lab_1/elk-stack-coursework/course-2-logstash/lab2 (main)
$ docker compose logs --since=15s logstash | grep -F 'order=79819'

PC@DESKTOP-SJHCAIT MINGW64 ~/Workspace/ELK/lab_1/elk-stack-coursework/course-2-logstash/lab2 (main)
$ |

```

We notice that after restarting the Logstash container, the event with the order 79819 is not read

Conclusion:

After restarting the Logstash container, previously processed lines were not read again. The old event is still visible when querying the complete Docker log history, but it does not appear in logs restricted to the period after the restart. This confirms that Logstash resumed from the saved file offset stored in `sincedb`

- **Observation after running the command:**
`docker compose down && docker compose up -d`

After `docker compose down` and `docker compose up -d`, previously processed events were replayed because the Logstash container was recreated and its `sincedb` state was lost. Since the input is configured with `start_position => "beginning"`, Logstash treated the log files as new and read them again from the beginning.

3.

Logstash's file input tracks the last-read offset of each file in `sincedb`. After a container restart, the same container and its `sincedb` state are preserved, so already-consumed lines are not read again; only new appended lines are processed. Old events may still appear when displaying the complete Docker log history, but they are not newly emitted. After `docker compose down` and `up`, offsets survive only if the Logstash data directory is persisted in a volume; otherwise files may be replayed. During development, a full replay can be forced with `sincedb_path => "/dev/null"` and `start_position => "beginning"`.

Mission 2: Modification of the pipeline file (nginx or web tag)

I parsed the nginx access logs using the `grok` filter to extract structured fields such as `clientip`, `method`, `request`, `status`, and `bytes`. I used the `date` filter to set `@timestamp` from the timestamp contained in the log line instead of the ingestion time. Finally, I removed the temporary fields (`raw_timestamp`, `ident`, `auth`) and the original `message` field using the `mutate` filter to keep the documents clean.

Mission 3: Modification of the pipeline file (app tag)

Both `Grok` and `Dissect` could parse the application logs. I chose `Dissect` because the log format is fixed and consistently separated by the `|` delimiter, making it simpler and faster than regular expressions. If the format changes or a delimiter is missing, `Dissect` leaves the event partially parsed, making parsing failures easy to detect and handle.

Mission 4: Modification of the pipeline file (orders tag)

```
ab2-logstash |      [0] "orders"
ab2-logstash |      ],
ab2-logstash |      "customer_country" => "IT",
ab2-logstash |      "path" => "/var/log/shopfast/orders.csv",
ab2-logstash |      "order_id" => "23655",
ab2-logstash |      "items" => 2,
ab2-logstash |      "@timestamp" => 2026-07-29T00:50:49.000Z,
ab2-logstash |      "host" => "686cdef47871"
ab2-logstash | }
```

Explanation:

I used the `csv` filter to parse the `orders.csv` file into structured fields: `order_id`, `ts`, `customer_country`, `items`, and `amount`. The `date` filter sets `@timestamp` from the `ts` column, while the `mutate` filter converts `items` to an integer and `amount` to a float. Finally, I removed the original `message` and `ts` fields to keep the output clean.

Modification 5:

I enriched the web events using the `geoip` filter to add geographical information based on the client IP, including the country and coordinates. I removed all `/healthz` requests with the `drop` filter so that internal monitoring traffic does not appear in the output. As a bonus, I used the `useragent` filter to extract browser, operating system, and device information from the user-agent string.

```

lab2-logstash |         "@version" => "1",
lab2-logstash |         "tags" => [
lab2-logstash |           [0] "web"
lab2-logstash |         ],
lab2-logstash |         "useragent" => {
lab2-logstash |           "os_major" => "17",
lab2-logstash |           "os_version" => "17.5",
lab2-logstash |           "os_name" => "iOS",
lab2-logstash |           "os_minor" => "5",
lab2-logstash |           "os_full" => "iOS 17.5",
lab2-logstash |           "os" => "iOS",
lab2-logstash |           "device" => "iPhone",
lab2-logstash |           "name" => "Mobile Safari UI/WKWebView"
lab2-logstash |         },
lab2-logstash |         "method" => "GET",
lab2-logstash |         "path" => "/var/log/shopfast/access.log",
lab2-logstash |         "status" => 502,
lab2-logstash |         "http_version" => "1.1"
lab2-logstash |       }
lab2-logstash |     {
lab2-logstash |       "user_agent" => "Mozilla/5.0 (iPhone; CPU iPhone OS 17_5 like Mac OS X) Mobile/15E148",
lab2-logstash |       "clientip" => "8.8.8.8",
lab2-logstash |       "bytes" => 20139,
lab2-logstash |       "@timestamp" => 2026-07-29T01:00:29.000Z,
lab2-logstash |       "request" => "/",
lab2-logstash |       "host" => "686cdef47871",
lab2-logstash |       "referrer" => "https://www.google.com/",
lab2-logstash |       "geoip" => {
lab2-logstash |         "location" => {
lab2-logstash |           "lat" => 37.751,
lab2-logstash |           "lon" => -97.822
lab2-logstash |         },
lab2-logstash |         "latitude" => 37.751,
lab2-logstash |         "longitude" => -97.822,
lab2-logstash |         "continent_code" => "NA",
lab2-logstash |         "country_code2" => "US",
lab2-logstash |         "ip" => "8.8.8.8",
lab2-logstash |         "country_name" => "United States",
lab2-logstash |         "timezone" => "America/Chicago",
lab2-logstash |         "country_code3" => "US"
lab2-logstash |       },

```

Mission 6: